App Playbook

How to build an app from an empty repo to launch, one part at a time. Each part says what good looks like, the skill that builds it, the skill that checks it, and how you know it is done.

Stack it assumes: Next.js (App Router) or React + Vite, shadcn/ui, Supabase or Clerk, Vercel, TypeScript, Vitest + Playwright, Sentry.

How to use it

1. Install the skills: `npx skills add stevetowers08/app-template-kit -g -a claude-code` (other options in the README).
2. Find the part you are on in the map below.
3. Tell Claude Code: **"Read the app playbook and take me through part 4."** (paste the playbook's GitHub link). It reads the part and runs the skills.

Two kinds of skill:

- **Setup skills** (`setup-...`) build something into your app.
- **Audit skills** (`audit-...` , `check-...`) check it, score it and list what to fix. They report first; only `audit-loading-animations` and `audit-design-system --fix` also edit code.

Run them one at a time, in order. Each one reads what the last one left behind.

The map

#	Part	Build with	Check with	Done when
1	Repo and agent setup	`setup-new-repo`	`audit-documentation` , `audit-repo` , `audit-architecture`	`npm run verify` is green in CI and a first page is live on a preview URL
2	Design system and UI	`shadcn` , `setup-design-system` , `setup-motion`	`audit-design-system` , `audit-frontend-design` , `audit-component-size`	One real page uses only tokens and shared components
3	Plan the first feature	plan mode or OpenSpec, `research`	-	The plan names the command that proves it works
4	Database	`supabase` , `postgresql-table-design`	`audit-database`	Every table has row-level security and a test that a stranger is denied
5	Auth and permissions	`supabase` or Clerk	`audit-security` , `audit-database`	Each resource has a test proving another user cannot read or change it
6	Features and APIs	`setup-error-handling`	`audit-api` , `audit-typescript`	Every route and action validates input, checks permission and has a test
7	Testing	`setup-testing`	`audit-tests` , `check-testing`	All test layers run green in CI; sign-in and payment flows have end-to-end tests

#	Part	Build with	Check with	Done when
8	Error handling	`setup-error-handling`	`audit-error-handling`, `audit-console-logs`	A forced error at each layer shows a friendly screen and gets logged
9	Loading and speed	`vercel-react-best-practices`	`audit-loading-animations`, `audit-page-loading`, `audit-performance`	Key pages hit LCP 2.5s, INP 200ms, CLS 0.1 for real users
10	Monitoring	`setup-sentry`	`audit-observability`	A deliberate production error reaches Sentry and fires an alert
11	Security	-	`audit-security`, `audit-dependencies`	OWASP ASVS level 1 passes
12	Release	`deploy-to-vercel`	`audit-production-readiness`	You have promoted once, rolled back once and restored a backup once
13	Landing page	`build-landing-page`, `setup-landing-page-mockup`, `product-demo-video`	`audit-frontend-design`	The page loads fast and every claim on it is true
14	After launch	`update-skills`	`check-cleanup`, `audit-dependencies`, `audit-documentation`, `audit-skill-freshness`	Dependencies, docs and skills are current

Skills in plain text with no link (`shadcn`, `supabase` and so on) are other people's. Install commands are in Other people's skills.

1. Repo and agent setup

When: the repo exists (empty or a cloned template) and an agent is about to work in it.

- Write `AGENTS.md` with only what an agent cannot work out from the code: exact commands, traps, decisions. Keep it under 200 lines. Hand-written beats generated.
- One `verify` command that runs everything (types, lint, tests, build). Done means verify is green.
- Set up CI that runs verify on every pull request.
- Deploy a thin first page early, so every later change ships to a preview URL.

Build: `/setup-new-repo` (add `--full` for the complete docs tree on a greenfield repo).

Check: `/audit-documentation`, `/audit-repo`, `/audit-architecture`.

2. Design system and UI

When: before you build screens. Changing the look later costs far more.

- Pick one look for the whole app. Browse the templates and Latent UI looks in this kit (`npm start`).
- Colours, spacing and radius come from CSS variables (tokens), never hard-coded values.
- Build with shared components (button, card, input). Never hand-roll one as a raw div.
- One element owns vertical scroll in the app shell.
- Generate `docs/design/` from the real code once tokens and components exist, not before.

Build: `shadcn` to install components, then `/setup-design-system` , then `/setup-motion` .

For taste: `interface-design` , `web-design-guidelines` .

Check: `/audit-design-system` , `/audit-frontend-design` (screenshots real pages), `/audit-component-size` .

3. Plan the first feature

When: before writing feature code.

- Pick the smallest slice that is useful on its own.
- Write the command that proves it works first. If you cannot write it, the feature is not clear yet.
- Small changes: plan mode. Multi-file or data-correctness work: a written spec (OpenSpec).
- Unfamiliar tech? Run `/research <topic>` first.

4. Database

When: before auth and features. The schema is the hardest thing to change later.

- Turn on row-level security (RLS) for every table. Supabase does not do it for you.
- Change the schema only through migrations, never by hand in production.
- Index foreign keys and anything you filter or sort on.
- Watch for N+1 queries (one query per row). Batch them.
- Rename or remove columns in three steps: add the new one, move the data and code, then drop the old one.

Build: `supabase` , `postgresql-table-design` , `supabase-postgres-best-practices` .

Check: `/audit-database` (runs the Supabase security and performance advisors).

5. Auth and permissions

When: with the database, before features.

- Login (who you are) and permission (what you can do) are separate. Agents are bad at permission by default: go table by table and decide who can read and write each one.
- Check permission on the server in every route and server action. A check in the page or the proxy/middleware alone is not enough.
- Keep data access in a server-only layer.
- Enforce access twice: in the API and in the database (RLS).

Check: `/audit-security` , `/audit-database` .

6. Features and APIs

When: every feature.

- Validate every input at the boundary (Zod).
- Return one consistent error shape from every route.
- Put timeouts and retries on calls to outside services.
- Strict TypeScript. No `any` escape hatches.

Build: `/setup-error-handling` at your first API route (adds `withApiHandler` and the error helpers).

Check: `/audit-api`, `/audit-typescript`.

7. Testing

When: set it up at the first feature, then every feature adds tests.

- Write the failing test first, then the code.
- Unit tests for logic, integration tests for database and API, end-to-end (Playwright) for sign-in, payments and the core feature.
- Every bug becomes a test so it cannot come back.
- Agents write tests that cannot fail. Check that each test fails when the code is wrong.

Build: `/setup-testing`.

Check: `/audit-tests` (coverage and test quality), `/check-testing` (a full QA pass).

8. Error handling

When: from the first API route.

- Expected errors get a clear message and a retry ("Could not reach the mail service. Try again.").
- Unexpected errors go to a global fallback screen (`error.tsx`, `global-error.tsx`).
- Never show stack traces to users. Never swallow an error silently.
- Fail closed: when something breaks, deny access, never grant it.

Build: `/setup-error-handling`.

Check: `/audit-error-handling`, `/audit-console-logs` (opens every page and reads the console).

9. Loading and speed

When: once real pages exist, and again before launch.

- Measure real users first, then lab tools. Targets at the 75th percentile: LCP 2.5s or less, INP 200ms or less, CLS 0.1 or less.
- Skeletons for content that is loading, spinners for actions.
- No request waterfalls: fetch in parallel, stream slow parts behind Suspense.
- Keep the app shell and sidebar instant.

Build: `vercel-react-best-practices`.

Check: `/audit-loading-animations`, `/audit-page-loading` (route by route), `/audit-performance` (measures bundle size and speed).

10. Monitoring

When: before anyone else uses the app.

- Error tracking with source maps, so errors point at your real code.
- Logs carry a request ID and user ID.
- Alert on new errors. Do not wait for users to send screenshots.

Build: `/setup-sentry` (it asks for your Sentry org, project and region).

Check: `/audit-observability`.

11. Security

When: before launch, then about monthly.

- Secrets live in environment variables, never in client code (watch `NEXT_PUBLIC_`).
- Add a pre-commit secret scanner so keys never reach git.
- Rate-limit sign-in and expensive routes.
- Set security headers and a Content Security Policy.
- Check against the OWASP Top 10 (2025), then OWASP ASVS level 1.
- Bugs in business logic are security bugs, such as a free user getting paid features when something breaks.

Check: `/audit-security`, `/audit-dependencies` (known vulnerabilities and supply chain). For a deep scan: `deepsec`, Trail of Bits skills.

12. Release

When: before the first real users.

- CI on every pull request: lint, typecheck, test, build. Fail means no deploy.
- Every push gets a preview deployment. Test there, then promote.
- Run database migrations before the code that needs them.
- Know your rollback, and do one for practice.

Build: `deploy-to-vercel`, `vercel-cli-with-tokens`.

Check: `/audit-production-readiness` (the umbrella check), then the launch checklist.

13. Landing page

When: before launch.

- Brand tokens first, then sections. Avoid the generic AI-made look.
- Show the real product: a coded mockup or a demo video.
- Metadata, Open Graph images, sitemap and robots file.
- Only claims you can back up.

Build: `/build-landing-page`, `/setup-landing-page-mockup`, `/product-demo-video`.

Check: `/audit-frontend-design`.

14. After launch

When: ongoing.

- Update dependencies regularly, with a short wait after each release (a cooldown) to dodge bad publishes.
- Keep `AGENTS.md` and docs current. A stale doc steers the agent wrong without telling you.
- Clean out old files and finished plans.
- Keep your skills current.

Build: `/update-skills`.

Check: `/check-cleanup`, `/audit-dependencies`, `/audit-documentation`, `/audit-skill-freshness`.

Every new feature

1. **Check what you already know.** Past outcomes, repo rules, your notes.
2. **Can it be built?** Does the data exist? What will the query cost?
3. **Plan or spec.** Small: plan mode. Multi-file or data-correctness: a spec.
4. **Write the proof first.** The command that shows it works.
5. **Build** in small commits.
6. **Run verify.** Green or it is not done.
7. **Write the outcome.** What landed, what changed from the plan, what to watch.

The rule: the first time something breaks, fix it. The second time, add the test or check that stops it coming back, then fix it.

Launch checklist

Things launch checklists cover that the parts above do not:

- [ ] Backups on, and one restore tested
- [ ] Rate limits on sign-in, sign-up and expensive routes; CAPTCHA if abused
- [ ] Security headers, CSP, deployment protection on previews
- [ ] Accessibility: WCAG 2.2 AA, `jsx-a11y` lint rule
- [ ] SEO: metadata, Open Graph images, sitemap, robots
- [ ] Privacy policy, and cookie consent if you serve the EU
- [ ] Email: SPF, DKIM and DMARC on your domain; custom SMTP for auth emails
- [ ] Spend alerts on hosting and paid APIs
- [ ] An incident plan: who gets alerted and how to roll back

Where pros disagree

- **When to deploy.** Some deploy last. This playbook deploys a thin slice first, so every change ships through the same path.
- **Test shape.** Many unit tests or mostly integration tests. Both work; testing the money paths end to end is not optional.
- **When to add monitoring.** Day one or before launch. Before anyone else uses it is the latest.
- **How much spec.** Big up-front specs are a known trap. Keep them to one small slice.

All skills in this kit

Skill	Use it when	Part
`setup-new-repo`	Starting a repo or adding agent docs to one	1
`audit-documentation`	Checking agent docs and the docs layout	1, 14
`audit-repo`	Asking "does our agent setup actually work?"	1
`audit-architecture`	Checking folder structure against what AGENTS.md says	1
`setup-design-system`	Tokens and components exist and need writing down	2
`setup-motion`	The UI works but feels static	2
`audit-design-system`	Checking token use, icon sizes, cards, scroll	2
`audit-frontend-design`	Judging how pages actually look, desktop and mobile	2, 13
`audit-component-size`	Files or components feel too big	2
`research`	Any unfamiliar tech, or "what do the pros do?"	3, any
`audit-database`	Checking RLS, indexes, migrations, slow queries	4, 5
`audit-security`	Checking secrets, auth, permissions, payments	5, 11
`setup-error-handling`	Your first API route	6, 8
`audit-api`	Checking routes and server actions	6
`audit-typescript`	Checking type safety and compiler settings	6
`setup-testing`	Your first feature worth protecting	7
`audit-tests`	Checking coverage and test quality	7
`check-testing`	A full QA pass before a release	7, 12
`audit-error-handling`	Checking for swallowed errors and missing boundaries	8
`audit-console-logs`	Finding console errors on every page	8
`audit-loading-animations`	Checking skeletons, spinners, flashes of empty content	9
`audit-page-loading`	A page feels slow or janky	9
`audit-performance`	Measuring bundle size and speed	9
`setup-sentry`	Before anyone else uses the app	10
`audit-observability`	Checking errors, logs, traces and alerts	10

Skill	Use it when	Part
`audit-dependencies`	Checking vulnerabilities and supply chain	11, 14
`audit-production-readiness`	The final check before launch	12
`build-landing-page`	Building a marketing site or landing page	13
`setup-landing-page-mockup`	The landing page needs a live product shot	13
`product-demo-video`	You want a product demo video	13
`check-cleanup`	Clearing stale files, old plans, build junk	14
`audit-skill-freshness`	Checking skills for outdated advice	14
`update-skills`	Installing, updating or publishing these skills	14

Other people's skills

Install these from their own source. Add `-g -a claude-code` to install for every project.

Skill	Part	Install
`shadcn`	2	`npx skills add shadcn/ui --skill shadcn`
`web-design-guidelines`, `vercel-react-best-practices`, `deploy-to-vercel`, `vercel-cli-with-tokens`, `vercel-optimize`	2, 9, 12	`npx skills add vercel-labs/agent-skills`
`interface-design`	2	Dammyjay93/interface-design
`design-taste-frontend`	13	`npx skills add leonxlnx/taste-skill`
`supabase`, `supabase-postgres-best-practices`	4, 5	`npx skills add supabase/agent-skills`
`postgresql-table-design`	4	`npx skills add wshobson/agents --skill postgresql-table-design`
`playwright-skill`	7	`npx skills add lackeyjb/playwright-skill --skill playwright-skill`
OpenSpec	3	Fission-AI/OpenSpec
`deepsec`	11	vercel-labs/deepsec
Trail of Bits skills	11	trailofbits/skills
AI agent skills (evals, tracing, workflows)	any	stevetowers08/ai-agent-skills

Update them with `npx skills check` and `npx skills update`.

Sources

- Order of work: Continuous Delivery, Humble and Farley, Thoughtworks: first story, Thoughtworks Radar: spec-driven development
- Agent docs: Claude Code best practices, Claude Code memory, agents.md, Agent Skills spec
- Database and auth: Supabase production checklist, Supabase RLS, Next.js data security, OWASP Next.js cheat sheet
- Errors, speed, release: Next.js production checklist, Core Web Vitals, Vercel production checklist, Google SRE: reliable launches, Google SRE: data integrity
- Security: OWASP Top 10 2025, OWASP ASVS 5.0
- Testing shape: Martin Fowler: test shapes
- Launch checklist: WCAG 2.2, Gmail sender rules, OAIC privacy principles, DORA metrics